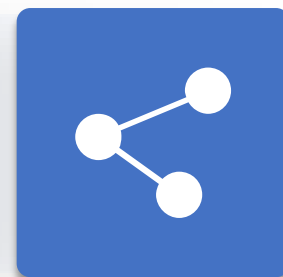




人工智能引论

第3章 有信息搜索

授课教师：李静瑶





录

1

贪心最佳优先搜索

2

A*搜索

3

内存受限搜索

4

启发式函数



最佳优先搜索

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** 一个解节点或 *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL) 评估函数 *f*

frontier $\leftarrow$ 一个以 *f* 排序的优先队列, 其中一个元素为 *node*

reached $\leftarrow$ 一个查找表, 其中一个条目的键为 *problem*.INITIAL, 值为 *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *s* 不在 *reached* 中 **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

将 *child* 添加到 *frontier* 中

return *failure*



$$\text{评估函数 } f(n) = \underbrace{g(n)}_{\text{当前实际代价}} + \underbrace{h(n)}_{\text{后续估计代价}}$$

- $g(n)$ 是从开始节点到当前节点 n 的**实际代价**
- $h(n)$ 是从当前节点 n 到目标节点的最优解的**估计代价**
- $f(n)$ 是**经过节点 n 的最优解的估计代价**

贪心最佳优先搜索



一致代价搜索: $f(n) = g(n)$

- 扩展路径代价 $g(n)$ 最小的节点
- $g(n)$ 是从开始节点到当前节点 n 的实际路径代价

贪心最佳优先搜索: $f(n) = h(n)$

- 扩展启发式函数 $h(n)$ 最小的节点
- $h(n)$ 是从当前节点 n 到目标节点的最小路径代价的估计值 (启发式函数),
若 n 为目标节点, 则 $h(n)=0$

贪心最佳优先搜索

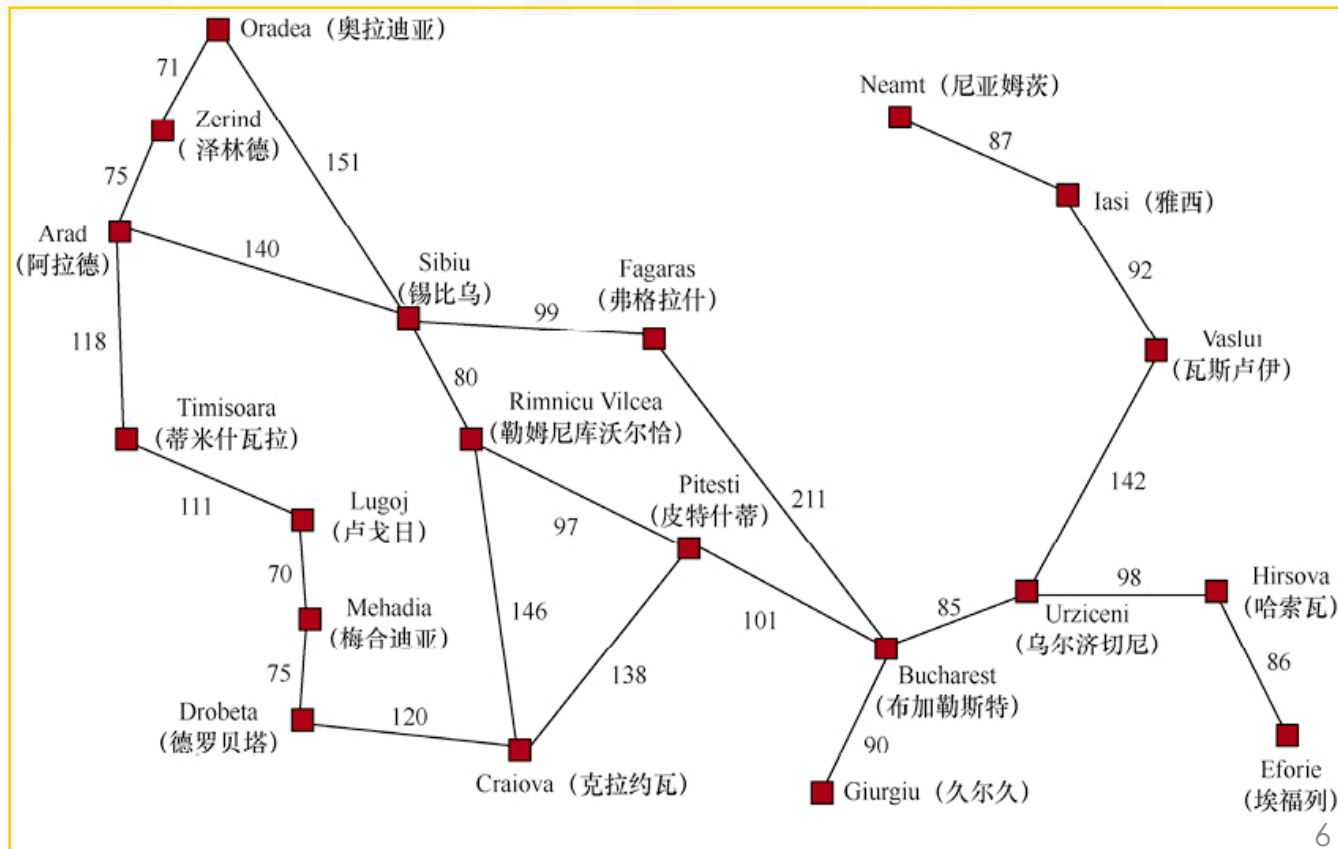


贪心最佳优先搜索

- 扩展看起来最接近目标的节点, $f(n) = h(n)$

到Bucharest的直线距离

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



贪心最佳优先搜索

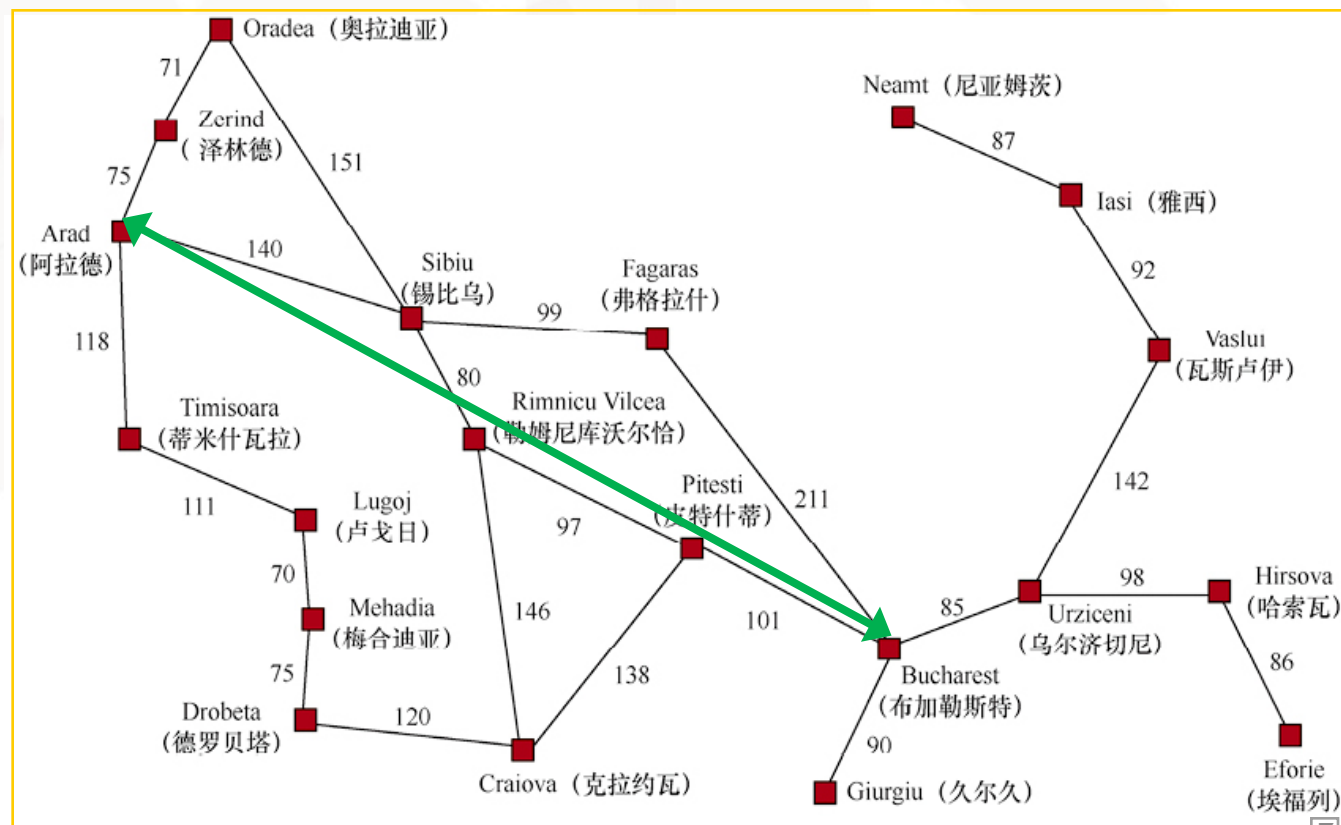


贪心最佳优先搜索

- 扩展看起来最接近目标的节点, $f(n) = h(n)$

到Bucharest的直线距离

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



贪心最佳优先搜索

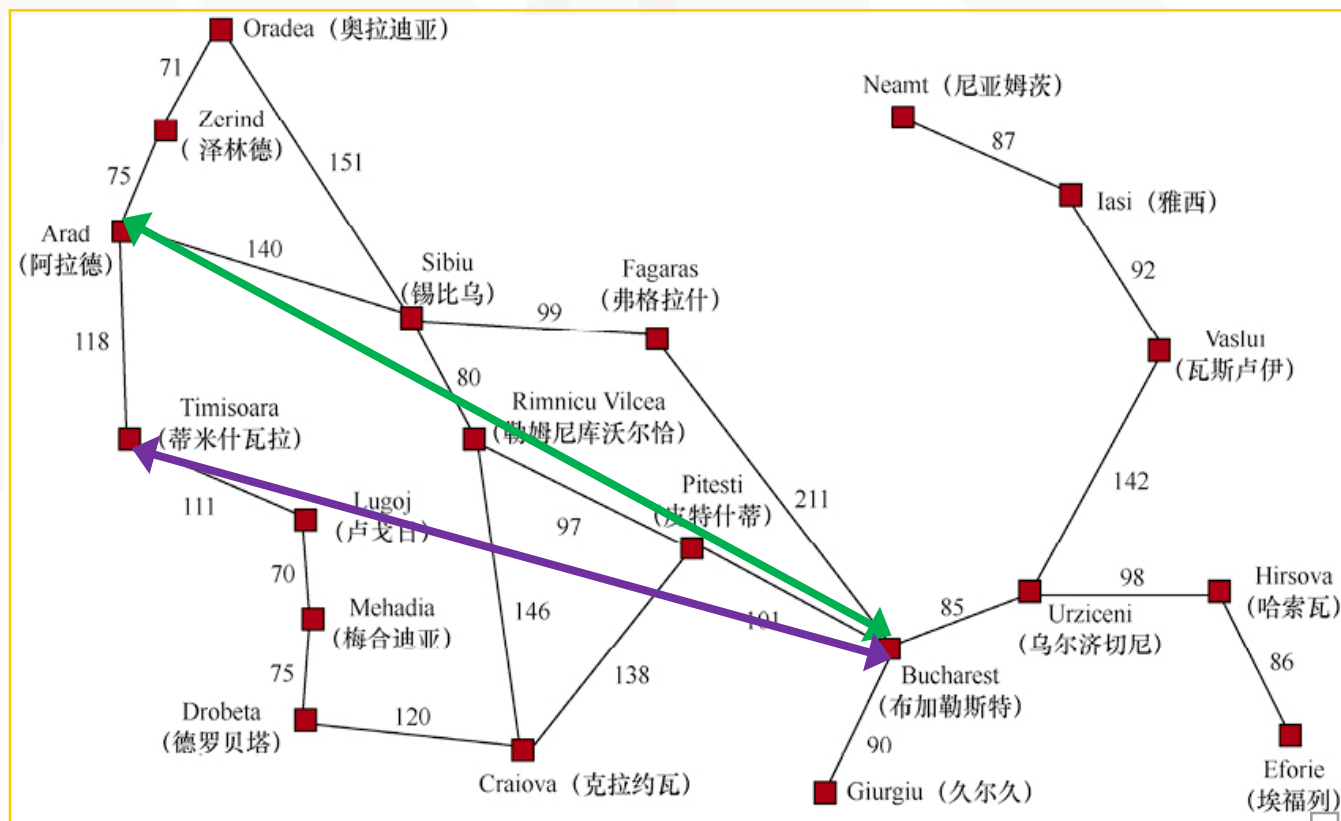


贪心最佳优先搜索

- 扩展看起来最接近目标的节点, $f(n) = h(n)$

到Bucharest的直线距离

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



贪心最佳优先搜索



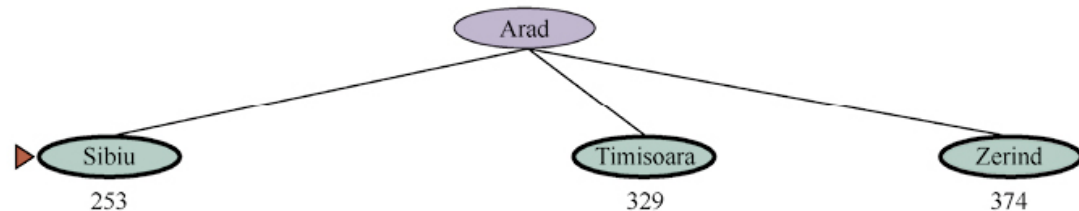
到Bucharest的直线距离

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

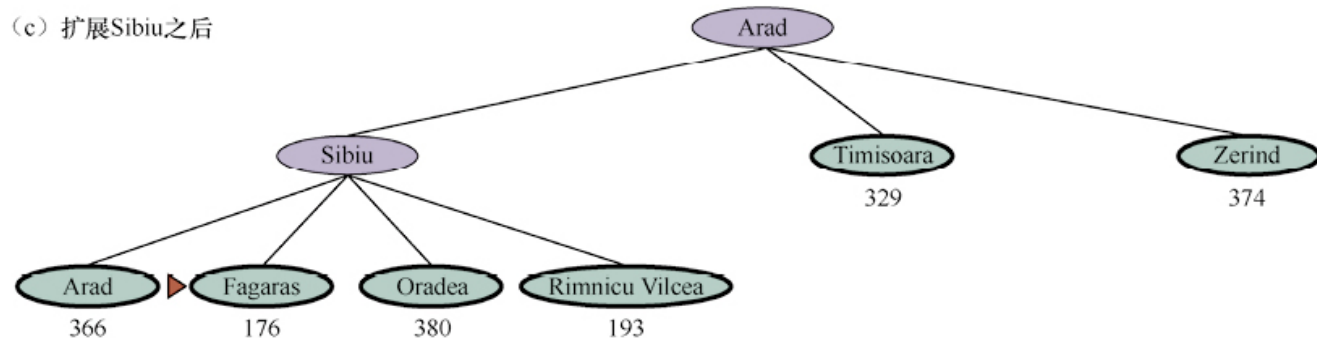
(a) 初始状态



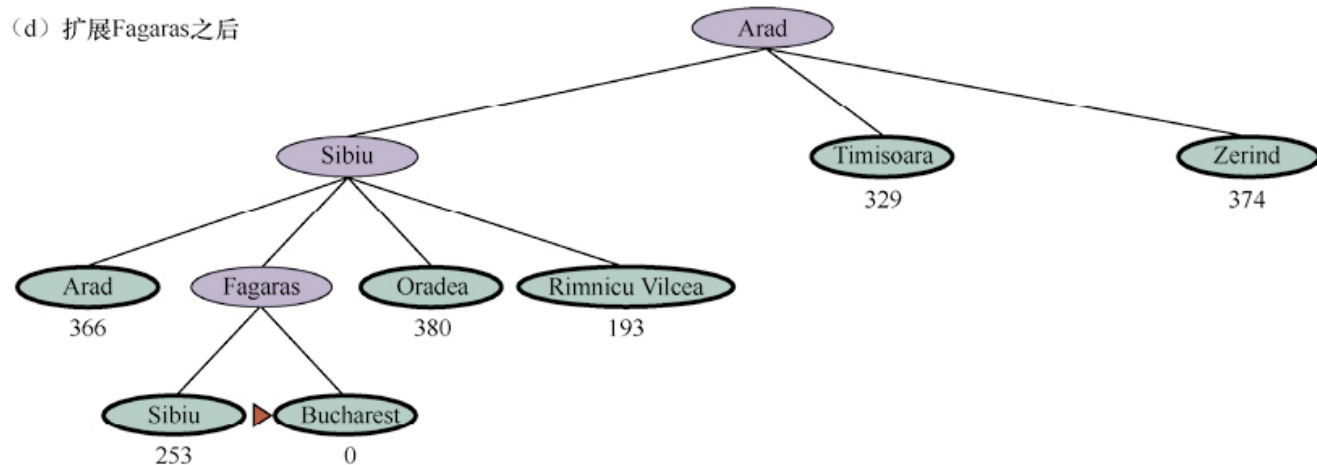
(b) 扩展Arad之后



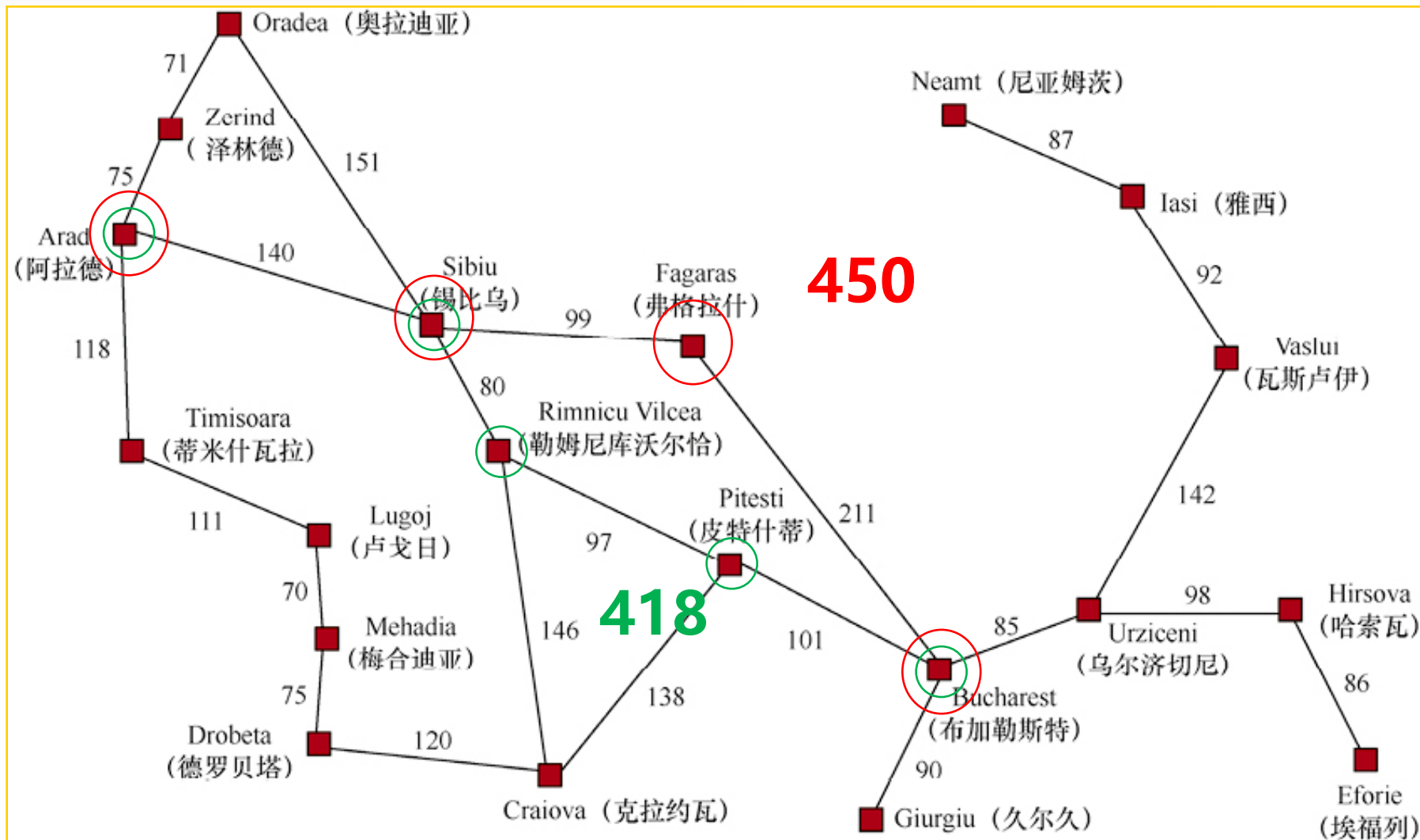
(c) 扩展Sibiu之后



(d) 扩展Fagaras之后



贪心最佳优先搜索

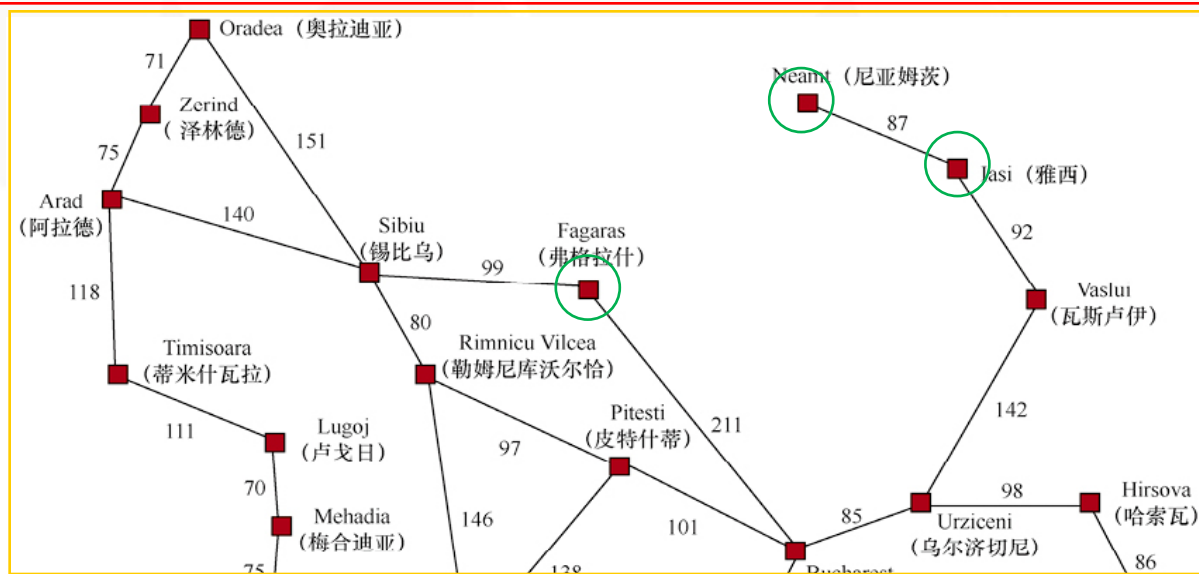


贪心最佳优先搜索

不足之处

- 贪心最佳优先搜索不是最优的
- 启发函数代价最小化这一目标会对错误的起点比较敏感
- 贪心最佳优先搜索有限状态空间完备，无限状态空间不完备
- 可能沿着一条无限的路径走下去而不回来做其它的选择尝试，因此无法找

到最佳路径这一答案





$$\text{评估函数 } f(n) = \underbrace{g(n)}_{\text{当前实际代价}} + \underbrace{h(n)}_{\text{后续估计代价}}$$

- $g(n)$ 是从开始节点到当前节点 n 的**实际代价**
- $h(n)$ 是从当前节点 n 到目标节点的最优解的**估计代价**
- $f(n)$ 是经过节点 n 的最优解的**估计代价**

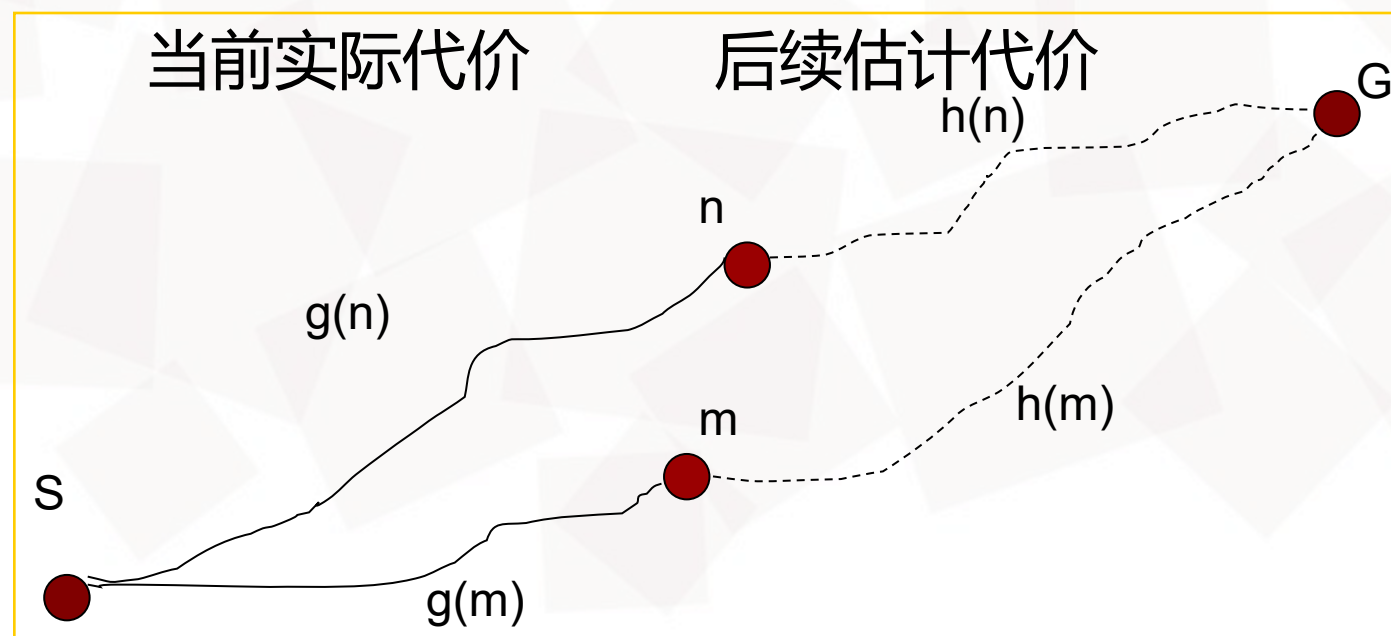
评估函数

 $f(n)$

=

 $g(n)$

+

 $h(n)$ 

比较 $f(n)$ 和 $f(m)$ 大小，决定节点搜索顺序，即在
Frontier表中的顺序



A*搜索：一种最佳优先搜索

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** 一个解节点或 *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ 一个以 *f* 排序的优先队列, 其中一个元素为 *node*

reached $\leftarrow$ 一个查找表, 其中一个条目的键为 *problem*.INITIAL, 值为 *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

s $\leftarrow$ *child*.STATE

if *s* 不在 *reached* 中 **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

将 *child* 添加到 *frontier* 中

return *failure*

A*搜索

function BEST-FIRST-SEARCH(*problem*, *f*) **returns** 一个解节点或 *failure*

node $\leftarrow$ NODE(STATE=*problem*.INITIAL)

frontier $\leftarrow$ 一个以 *f* 排序的优先队列, 其中一个元素为 *node*

reached $\leftarrow$ 一个查找表, 其中一个条目的键为 *problem*.INITIAL, 值为 *node*

while not IS-EMPTY(*frontier*) **do**

node $\leftarrow$ POP(*frontier*)

if *problem*.IS-GOAL(*node*.STATE) **then return** *node*

for each *child* **in** EXPAND(*problem*, *node*) **do**

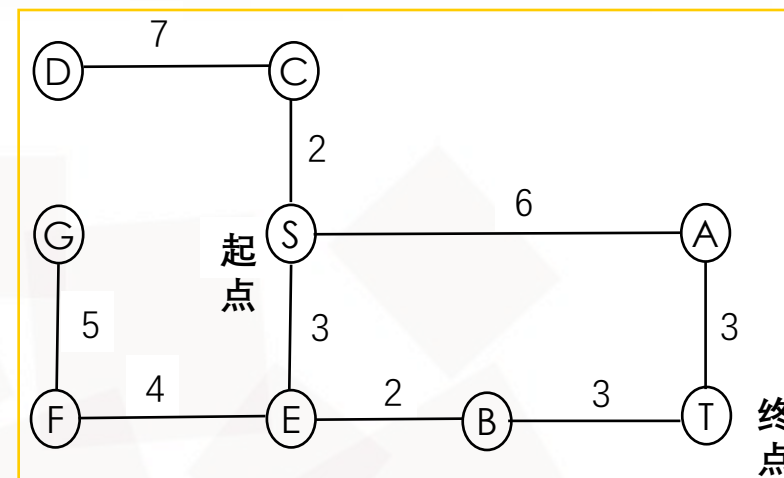
s $\leftarrow$ *child*.STATE

if *s* 不在 *reached* 中 **or** *child*.PATH-COST < *reached*[*s*].PATH-COST **then**

reached[*s*] $\leftarrow$ *child*

将 *child* 添加到 *frontier* 中

return *failure*



$h(S)=6$

$h(A)=3$

$h(B)=1$

$h(C)=8$

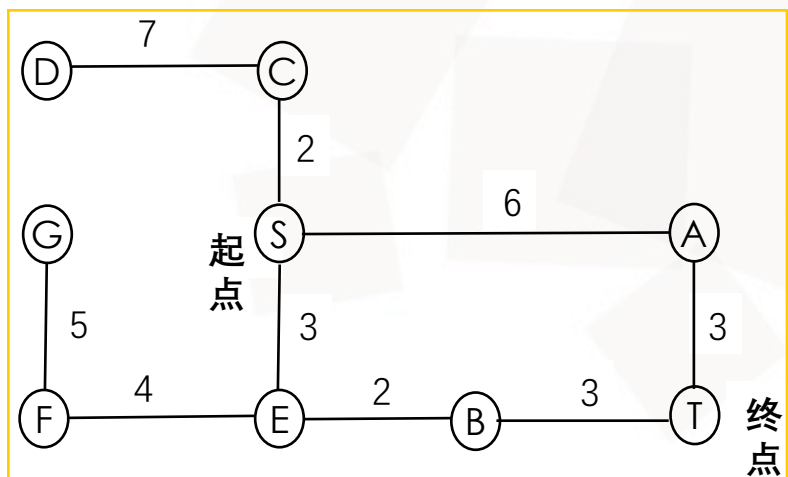
$h(D)=13$

$h(E)=4$

$h(F)=7$

$h(G)=12$

$h(T)=0$



$$h(S)=6$$

$$h(A)=3$$

$$h(B)=1$$

$$h(C)=8$$

$$h(D)=13$$

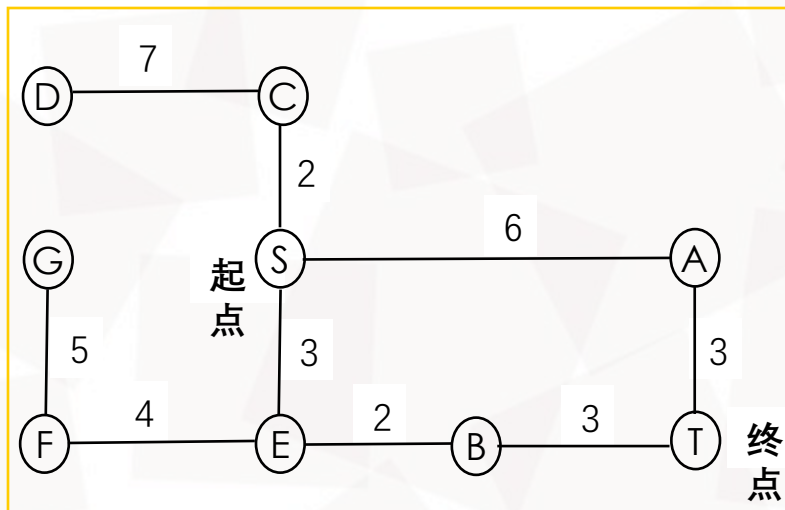
$$h(E)=4$$

$$h(F)=7$$

$$h(G)=12$$

$$h(T)=0$$

A*搜索



$$h(S)=6$$

$$h(A)=3$$

$$h(B)=1$$

$$h(C)=8$$

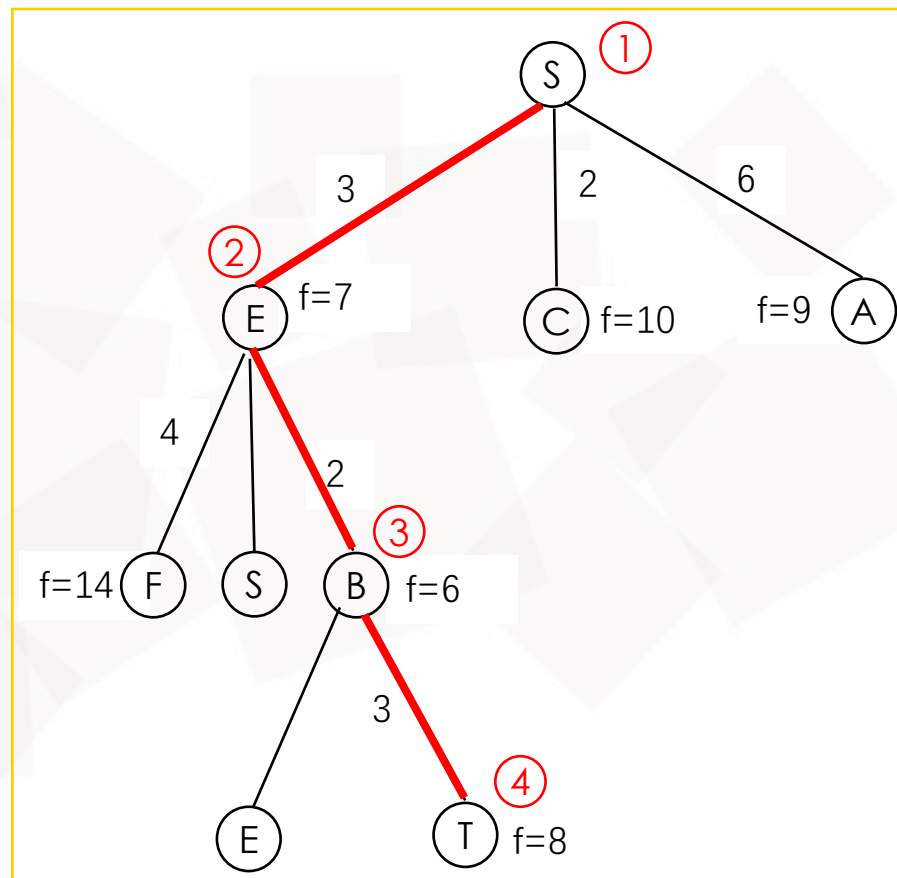
$$h(D)=13$$

$$h(E)=4$$

$$h(F)=7$$

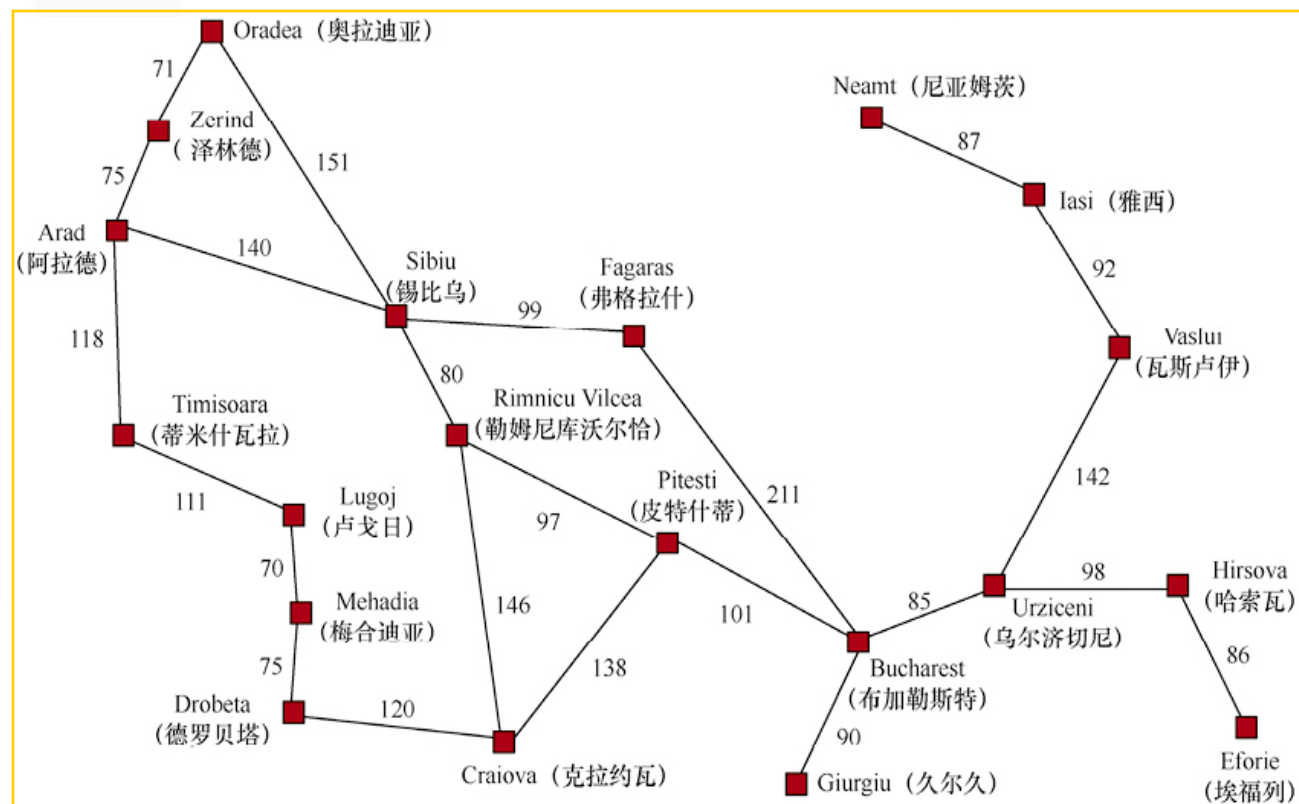
$$h(G)=12$$

$$h(T)=0$$



$$A^*: f(n)=g(n)+h(n)$$

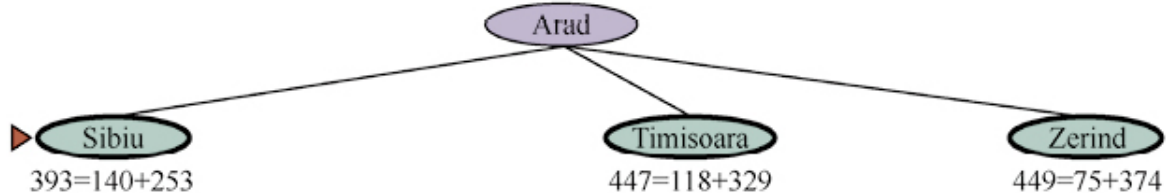
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



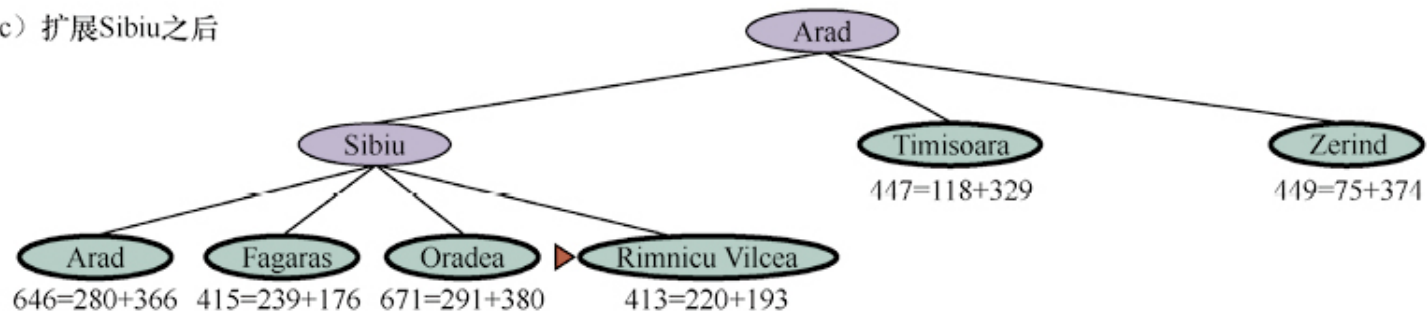
(a) 初始状态



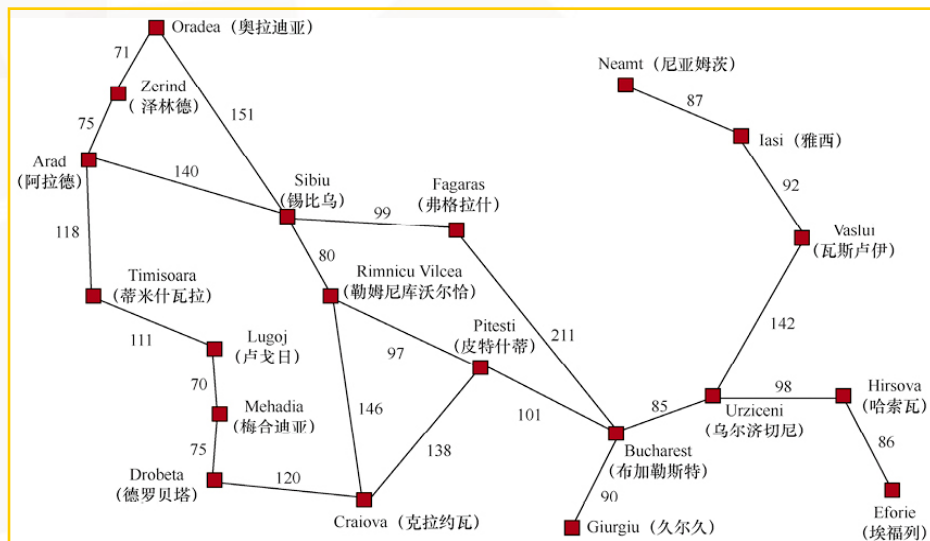
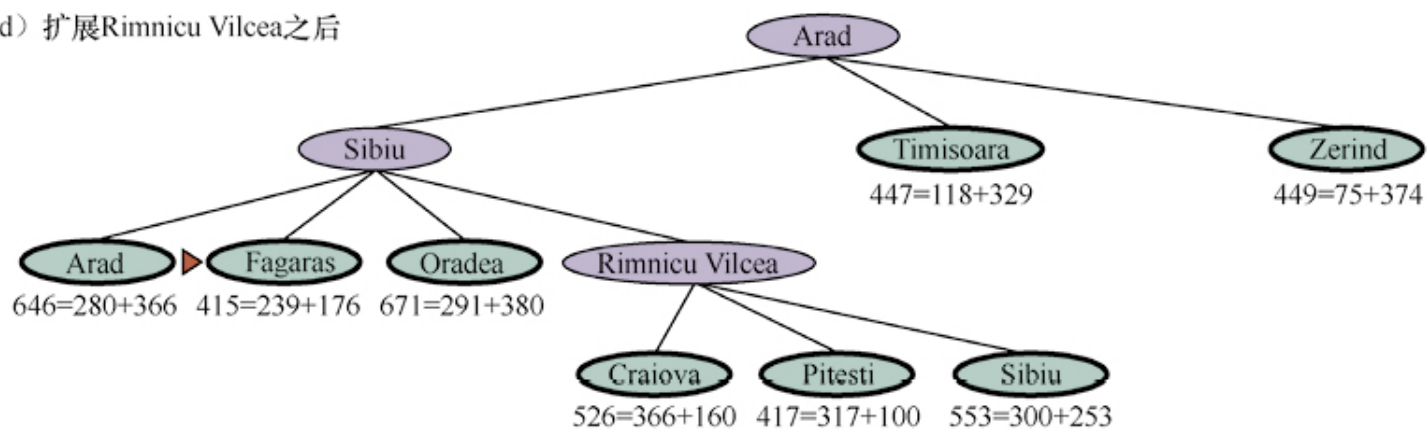
(b) 扩展Arad之后



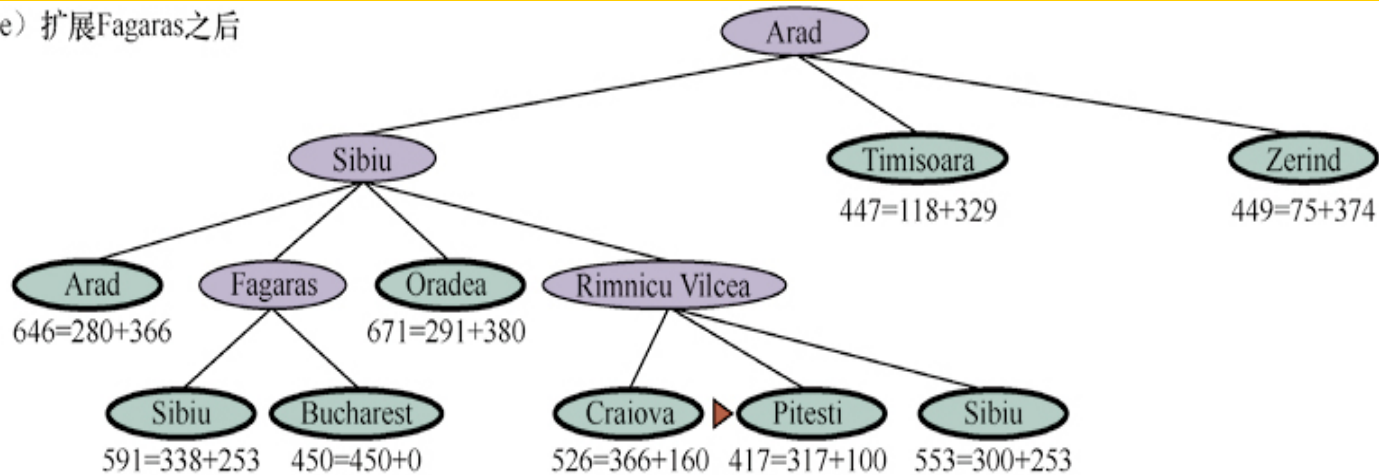
(c) 扩展Sibiu之后



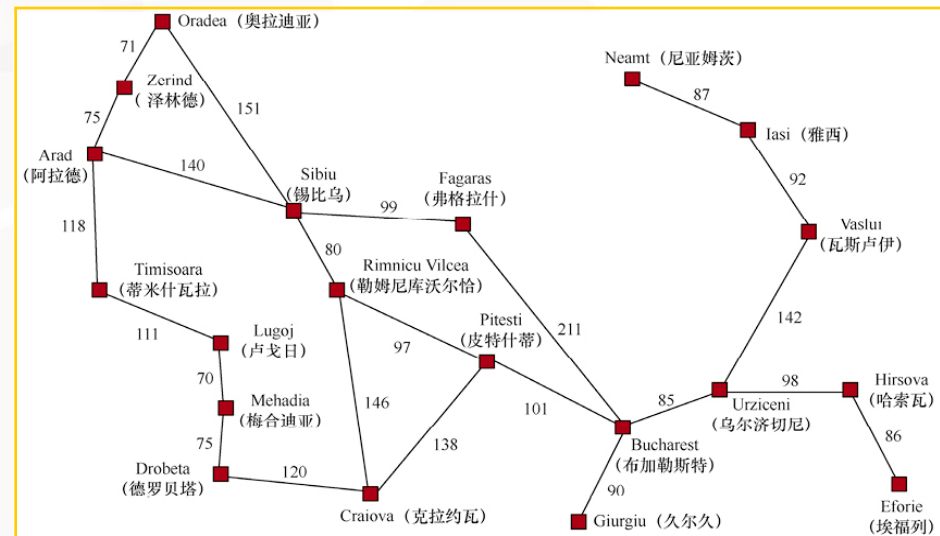
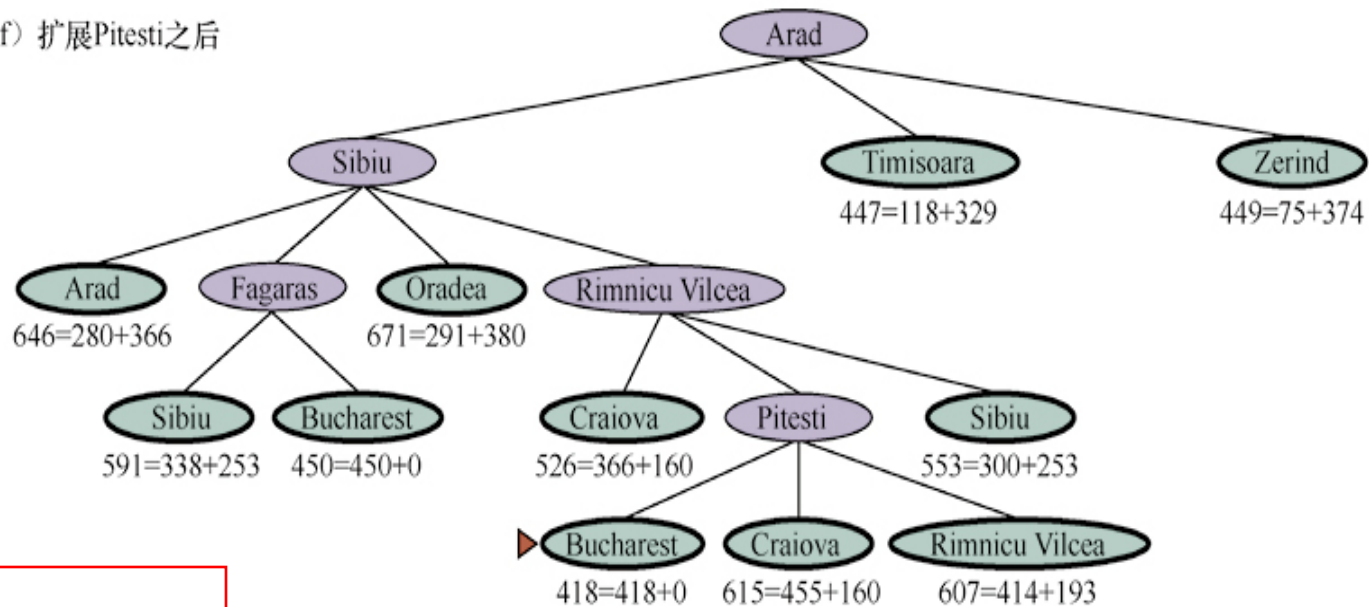
(d) 扩展Rimnicu Vilcea之后



(e) 扩展Fagaras之后



(f) 扩展Pitesti之后





可容许性:

- 函数永远不会高估到达某个目标的代价 $h(n) \leq h^*(n)$
- $f(n)$ 不会超过经过节点 n 的最优解的实际代价
- 若 $h(n)$ 是可容许的, 则A搜索被称为A*搜索, 具有最优性



可容许性:

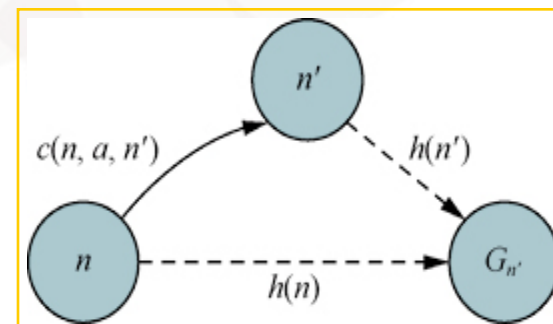
反证法证明最优性。假设最优路径的代价为 C^* ，但返回 $C > C^*$ ，那么最优路径上一定存在节点 n 未扩展。 $g^*(n)$ 表示从起点到 n 的最优路径代价， $h^*(n)$ 表示从 n 到最近目标的最优路径的代价。

$f(n) > C^*$	否则 n 将是已扩展节点
$f(n) = g(n) + h(n)$	根据定义
$f(n) = g^*(n) + h(n)$	因为 n 在一条最优路径上
$f(n) \leq g^*(n) + h^*(n)$	根据可容许性, $h(n) \leq h^*(n)$
$f(n) \leq C^*$	根据定义, $C^* = g^*(n) + h^*(n)$

第一行与最后一行矛盾，“算法返回次优路径”的假设是错误的。

一致性:

- 对于任意节点 n 和通过动作 a 生成的后继结点 n' , 从 n 到达目标的估计代价不大于从 n 到 n' 的单步代价与从 n' 到达目标的估计代价之和: $h(n) \leq c(n, a, n') + h(n')$
- 从 n 到达目标的估计代价是所有从 n 到达目标的代价的下界





启发式函数——直线距离：具有可容许性和一致性

- 将直线距离作为启发函数 $h(n)$ ，则启发函数一定是可容的，因为其不会高估开销代价
- $g(n)$ 是从起点到节点 n 的实际代价开销，且 $f(n)=g(n)+h(n)$ ，因此 $f(n)$ 不会高估经过节点 n 路径的实际开销
- $h(n) \leq c(n, a, n') + h(n')$ 构成三角不等式，满足一致性



递归最佳优先搜索

- 模拟最佳优先搜索，但仅使用线性空间
- f_limit : 从历史扩展节点得到的最优备选路径的 f 值
- 若当前节点的 f 值超过了 f_limit ，递归将回到备选路径上
- 递归回溯时，用当前节点的子节点的最佳 f 值替换更新当前节点的 f 值，能记住遗忘的最佳子节点，用于决定以后是否需要重新扩展该子树



递归最佳优先搜索

function RECURSIVE-BEST-FIRST-SEARCH(*problem*) **returns** 一个解或者*failure*

solution, fvalue $\leftarrow$ RBFS(*problem*, NODE(*problem*.INITIAL), ∞)

return *solution*

function RBFS(*problem, node, f_limit*) **returns** 一个解或*failure*, 以及一个新的 *f* 代价限制

if *problem*.Is-GOAL(*node*.STATE) **then return** *node*

successors $\leftarrow$ LIST(EXPAND(*node*))

if *successors* 为空 **then return** *failure, ∞*

for each *s* **in** *successors* **do** //用前一次搜索中的值更新 *f*

s.f $\leftarrow$ max(*s*.PATH-COST + *h*(*s*), *node.f*)

while true do

best $\leftarrow$ *successors* 中 *f* 值最低的节点

if *best.f* > *f_limit* **then return** *failure, best.f*

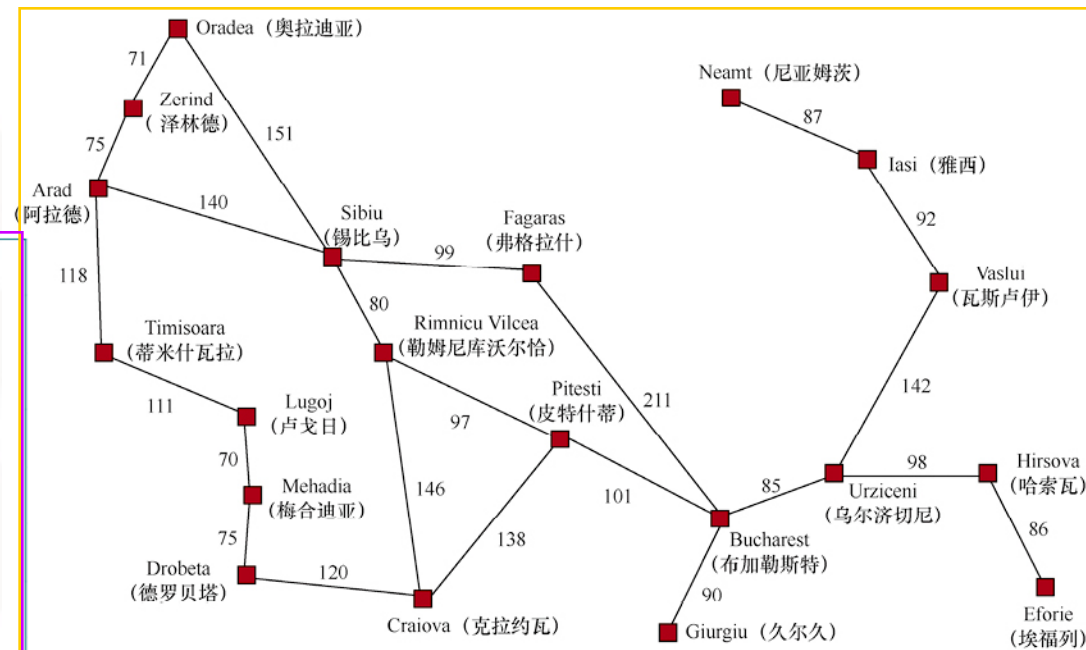
alternative $\leftarrow$ *successors* 中第二低的 *f* 值

result, best.f $\leftarrow$ RBFS(*problem, best, min(f_limit, alternative)*)

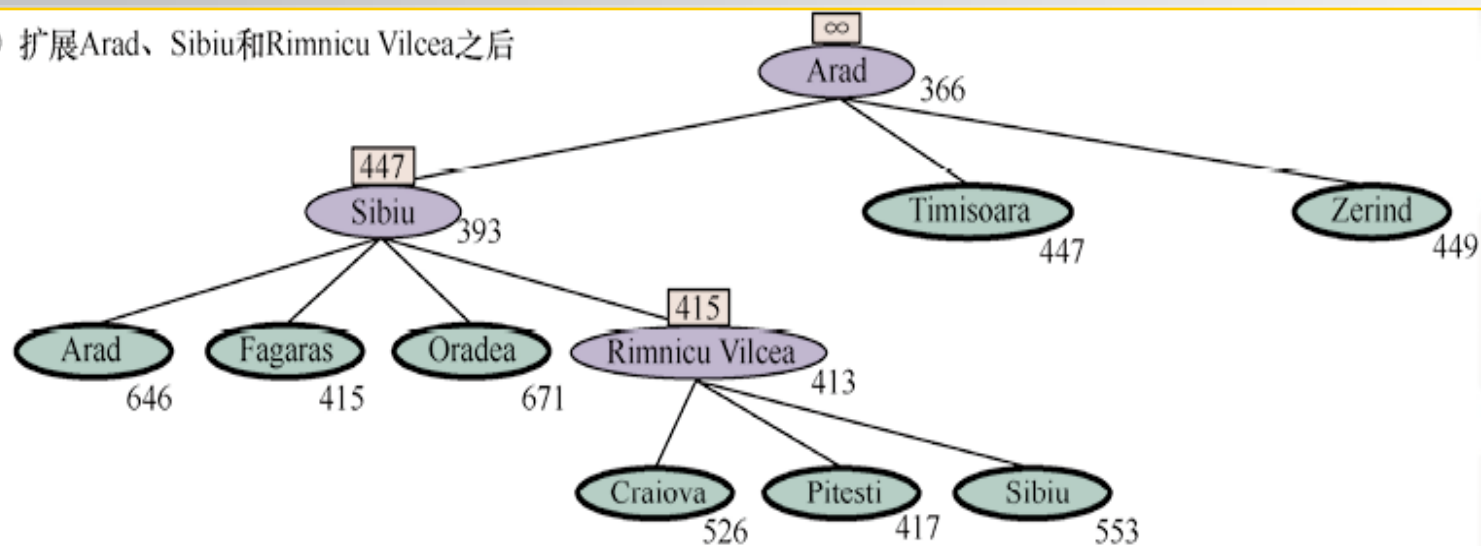
if *result* $\neq$ *failure* **then return** *result, best.f*



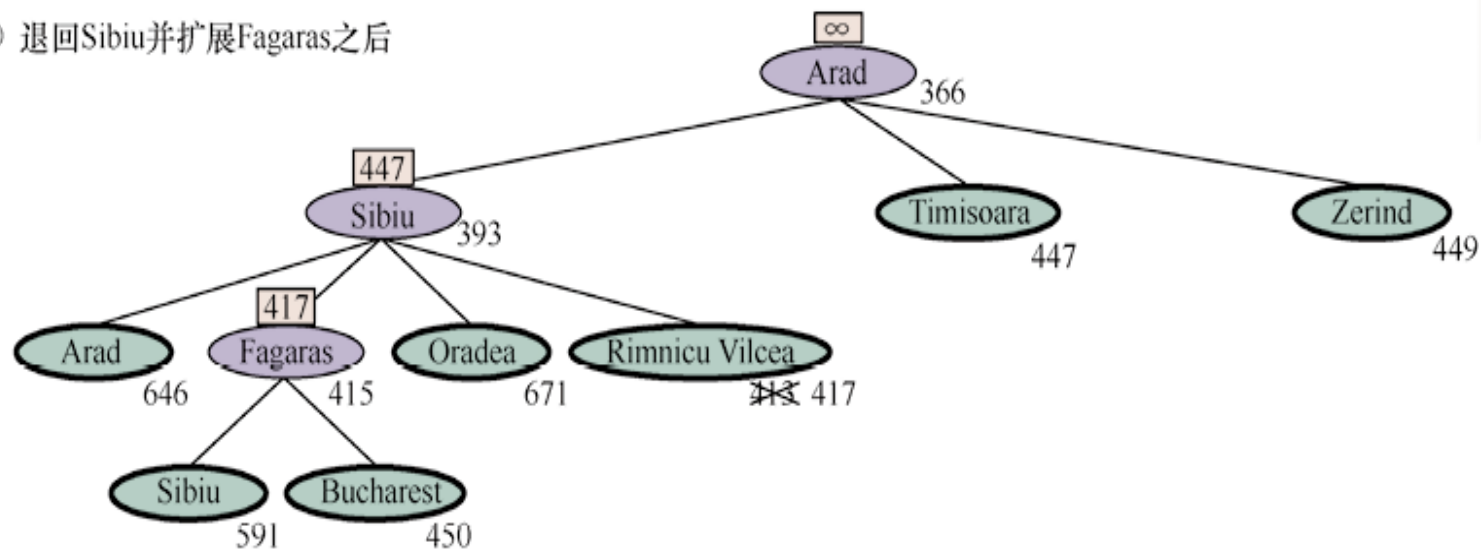
Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374



(a) 扩展Arad、Sibiu和Rimnicu Vilcea之后

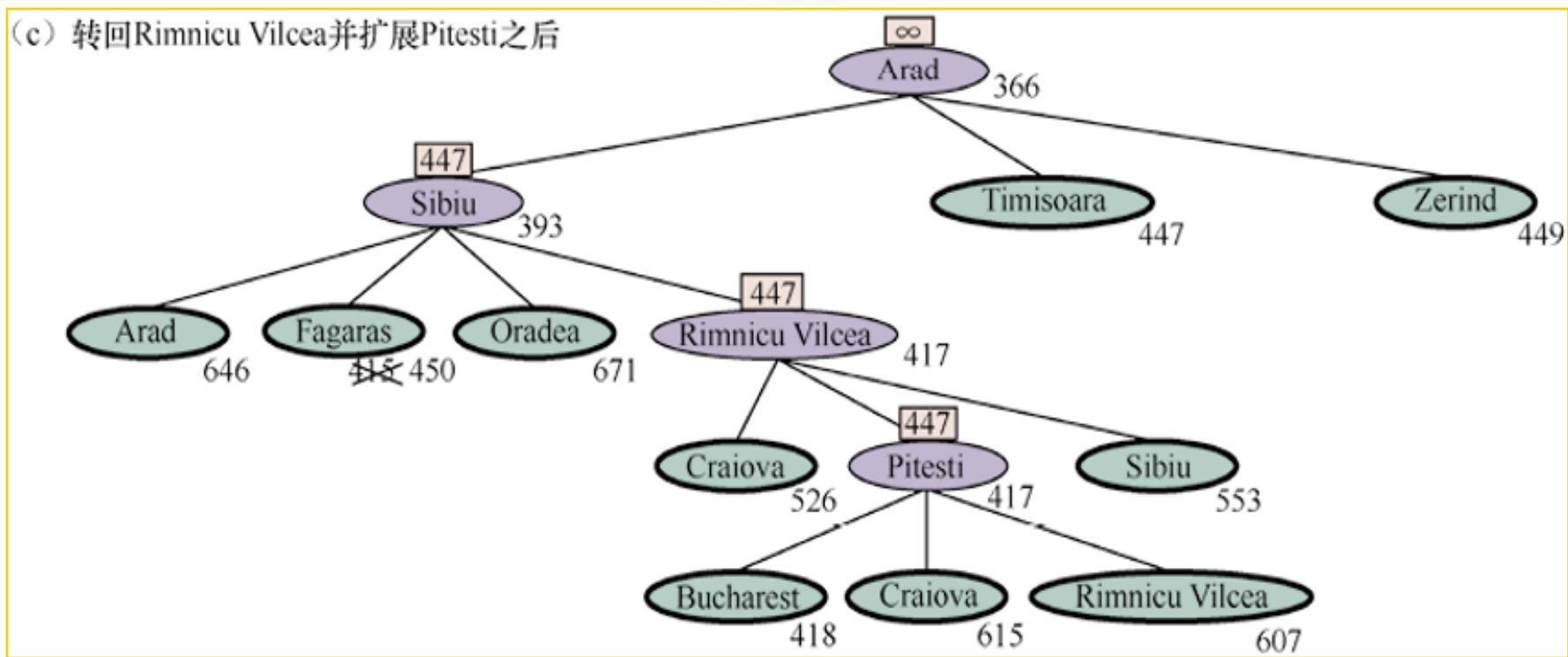


(b) 退回Sibiu并扩展Fagaras之后





(c) 转回Rimnicu Vilcea并扩展Pitesti之后





- 启发式函数的准确性
- 利用松弛问题设计启发式函数
- 利用子问题设计启发式函数

本节以八数码问题为例，探讨启发式函数的一般性质



启发式函数的准确性

- 描述启发式函数质量的方法：有效分支因子 b^*
- 若A*算法的解路径长度为d，生成结点数为N，则 b^* 是深度为d的均衡搜索树生成N+1个节点的有效分支因子，满足 $1+b^*+b^{*2}+\dots+b^{*d}=N+1$
- b^* 越小越好（N越小越好），最好情况为1
- 目标：设计启发式函数，使得 b^* 尽可能的小



例：八数码问题

- 一个 3×3 的棋盘上有8个数字滑块和一个空格。与空格相邻的滑块可以滑动到空格中（状态数181 400， 15数码的状态10万亿）
- 游戏目标是要达到一个特定的状态

7	2	4
5		6
8	3	1

开始状态

	1	2
3	4	5
6	7	8

目标状态

两个常用的启发式函数

- h_1 =不在位的滑块数 (8)
- h_2 =所有滑块到其目标位置的Manhattan距离和 (18)
- h_1 和 h_2 均为可容许的，但求解性能不同 (最优解26)

7	2	4
5		6
8	3	1

开始状态

	1	2
3	4	5
6	7	8

目标状态



d	搜索代价：生成的节点数			有效分支因子		
	BFS	$A^*(h_1)$	$A^*(h_2)$	BFS	$A^*(h_1)$	$A^*(h_2)$
6	128	24	19	2.01	1.42	1.34
8	368	48	31	1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17 270	1683	364	1.74	1.48	1.32
18	41 558	4102	751	1.72	1.49	1.34
20	91 493	9905	1318	1.69	1.50	1.34
22	175 921	22 955	2548	1.66	1.50	1.34
24	290 082	53 039	5733	1.62	1.50	1.36
26	395 355	110 372	10 080	1.58	1.50	1.35
28	463 234	202 565	22 055	1.53	1.49	1.36



占优

- 对于任意结点 n , $h_2(n) \geq h_1(n)$, 称 h_2 比 h_1 占优
- 使用 h_2 的A*永远不会比使用 h_1 的A*扩展更多的节点
- 每个满足 $f(n) < C^*$ 的节点 n 必将被扩展, 且 $f(n) = g(n) + h(n)$, 即每个满足 $h(n) < C^* - g(n)$ 的节点 n 必将被扩展
- 采用 h_2 的A*搜索扩展的结点必将被采用 h_1 的A*搜索扩展



从松弛问题出发设计启发式函数

- 松弛问题：减少动作限制的问题
 - ▶ 八数码问题中，若允许滑块任意移动位置，而不只是移动到相邻位置，则 h_1 是最短解的准确长度
 - ▶ 八数码问题中，若允许滑块向任意方向移动，甚至移动到被其它滑块占据的位置，则 h_2 是最短解的准确长度

7	2	4
5		6
8	3	1

开始状态

	1	2
3	4	5
6	7	8

目标状态



原问题与松弛问题的关联

- 减少动作限制导致状态空间图中边可能增加，故减少动作限制的松弛问题的状态空间图是原问题的状态空间图的超图
- 原问题的最优解是松弛问题的候选解，未必是松弛问题的最优解
- 松弛问题由于增加的边可能导致捷径，可能存在更好的解
- 松弛问题的最优解代价可以用作设计原问题的一个可容许的启发式函数



例：八数码问题的松弛问题

- 原问题：如果方格X与方格Y相邻，且Y是空格，那么滑块可以从方格X移动到方格Y
- 三个松弛问题：
 - 如果方格X与方格Y相邻，那么滑块可以从方格X移动到方格Y
 - 如果方格Y是空格，那么滑块可以从方格X移动到方格Y
 - 滑块可以从方格X移动到方格Y



新的启发式函数的构造:

- 可容许的启发式集合 $h_1, h_2, h_3, \dots$ 可以求解同一个问题, 但差别不明显

$$h(n) = \max \{h_1(n), \dots, h_k(n)\}$$

- $h_1, h_2, h_3, \dots$ 均可容许, 则 h 可容许
- h 比 $h_1, h_2, h_3, \dots$ 均占优

利用子问题的最优解代价设计启发式函数

*	2	4
*		*
*	3	1

开始状态

	1	2
3	4	*
*	*	*

目标状态



基于模式数据库设计启发式函数

- 对所有可能的子问题实例存储解代价，建立模式数据库
- 对于原问题的每个完备状态，通过在模式数据库中查找相应的子问题，返回子问题的最优解代价作为启发式函数值
- 每个数据库对应一个可容许的启发式函数 h_{DB}
- 可以通过max设计新的可容许的启发式函数
- 不同子问题如果定义了不相交模式数据库，则启发式函数相加仍是可容许的



例：八数码问题

- 一个 3×3 的棋盘上有8个数字棋子和一个空格。与空格相邻的棋子可以滑动到空格中。
- 游戏目标是要达到一个特定的状态

4	3	1
	5	2
6	7	8

初始状态



	1	2
3	4	5
6	7	8

目标状态

例：八数码问题

- 评估函数 $f(n)=g(n)+h(n)$
- $g(n)$ 表示结点 n 目前经过的移动次数，即其在搜索树中的深度 $h(n)$ 表示每个数字棋子与目标位置的距离之和，满足可容许性和一致性

4	3	1
	5	2
6	7	8

初始状态



	1	2
3	4	5
6	7	8

目标状态